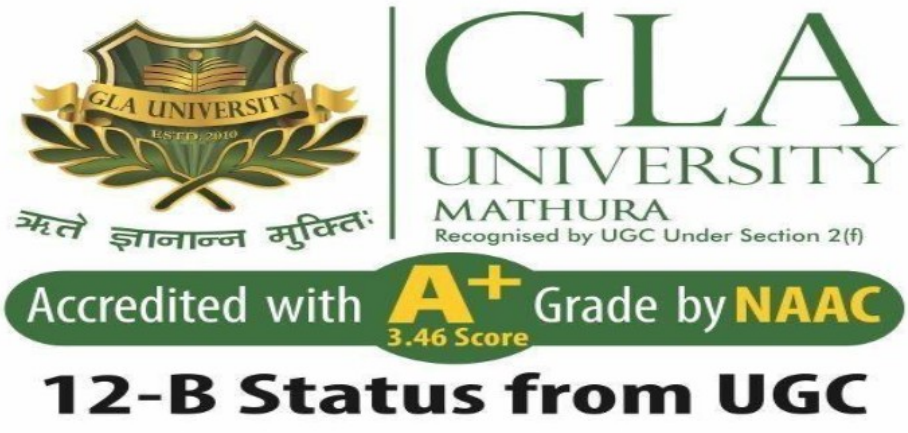
B.Tech CS (AI/ML) (III YEAR – V SEM) (2025–26)

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING & APPLICATIONS



GLA UNIVERSITY

17km Stone, NH-2, Mathura-Delhi Road, P.O. Chaumuhan, Mathura – 281406
(Uttar Pradesh) India

Project Title: eBPF-Shield AI: Kernel-Level Cloud-Native Runtime Threat & Container Escape Engine

Team Leader:	Arpit Sharma	UR: 2415500099
Team Member 1:	Aryan Gupta	UR: 2415500102
Team Member 2:	Ananya Mishra	UR: 2415500068
Mentor Name:	Miss Sanjana Shaw	

Signature :

Project Synopsis

eBPF-Shield AI: Kernel-Level Cloud-Native Runtime Threat & Container Escape Engine

Cover

Field	Details
Project title	eBPF-Shield AI: Kernel-Level Cloud-Native Runtime Threat & Container Escape Engine
Team	Arpit Sharma, Aryan Gupta, Ananya Mishra
Institute / course	GLA University, B.Tech CS (AI/ML)
Domain	eBPF, Kubernetes, Container Security, Machine Learning
GitHub Repository	https://github.com/PILOT-SHARMA/ebpf-shield-ai
Version / date	v1.0 / September 2026

1. Overview

- Problem: Container attacks are difficult to detect using normal application-level monitoring.
- Goal: Monitor container activity from the Linux kernel and detect suspicious behavior quickly.
- Main idea: Use eBPF for monitoring, machine learning for anomaly detection and a dashboard for alerts.
- Non-goal: The project will not replace a complete commercial security platform.

Value proposition: eBPF-Shield AI combines low-overhead kernel monitoring, explainable anomaly detection and Kubernetes-aware response in one academic platform.

2. Scope and Control

2.1 In-scope

- Process, file and network monitoring using eBPF.
- Kubernetes pod and container identification.
- Rule-based and ML-based threat detection.
- Live dashboard, alerts and approved response actions.

2.2 Out-of-scope

- Windows and macOS monitoring.
- Full packet-content inspection.
- Automatic destructive action without approval.

2.3 Assumptions

- The test system supports eBPF.
- Kubernetes information is available.
- Normal workload data is available for ML training.

2.4 Constraints

- Limited project time and attack data.
- Different Linux kernel versions.
- Possible false alerts from ML models.

2.5 Dependencies

- Linux kernel and eBPF tools.
- Docker and Kubernetes.
- FastAPI, PostgreSQL and Next.js.
- Python ML libraries.

2.6 Acceptance Criteria and Sign-off

- Live events should appear within about three seconds.
- Selected attack tests should generate alerts.
- Response actions should require approval and be logged.
- Sensor overhead should remain within the tested limit.

Stakeholder	Decision area	Approval	Date
Miss Sanjana Shaw - Mentor	Scope and final acceptance	_____	_____
Arpit Sharma - Team Lead	Release readiness	_____	_____
Department Reviewer	Academic submission	_____	_____

3. Stakeholders and RACI

R = Responsible, A = Accountable, C = Consulted, I = Informed.

Activity	Mentor	Arpit	Aryan	Ananya	Department
Requirements	C	A/R	C	C	I
Architecture and design	C	A/R	R	C	I
eBPF and backend	I	A	R	C	I
ML detection	C	A/R	C	I	I
Frontend and UX	I	A	C	R	I
Testing and documentation	C	A	C	R	I
Release and submission	C	A/R	C	C	I

4. Team and Roles

Member	Role	Responsibilities	Key skills	Availability	Contact
Arpit Sharma	Team Lead / ML	Scope, architecture, anomaly models, integration and reviews	Python, ML, security, Kubernetes	10 hrs/week	arpit.sharma_cs.aiml24@gla.ac.in

Member	Role	Responsibilities	Key skills	Availability	Contact
Aryan Gupta	eBPF / Backend	Kernel sensors, collector, FastAPI, database and WebSockets	C/eBPF, Python, APIs, PostgreSQL	10 hrs/week	aryan.gupta_cs.aiml24@gla.ac.in
Ananya Mishra	Frontend / QA	Dashboard, UX, testing, documentation and demo support	React, TypeScript, testing, writing	8 hrs/week	ananya.mishra_cs.aiml24@gla.ac.in

5. Week-wise Plan and Assignments

Week / dates	Milestone	Arpit	Aryan	Ananya	Deliverable	Status
1 / 7-13 Sep	Requirements	Finalize scope and metrics	Study eBPF feasibility	User flow and test plan	SRS and backlog	Planned
2 / 14-20 Sep	Architecture	Threat model and ML plan	Probe and API design	UI wireframes	Architecture, ERD, API v1	Planned
3 / 21-27 Sep	Telemetry	Feature specification	Process/file/network collector	Live-events screen	Telemetry demo	Planned
4 / 28 Sep-4 Oct	Backend	Detection rules	FastAPI, enrichment, database	Alert and detail screens	Integrated backend	Planned
5 / 5-11 Oct	ML detection	Isolation Forest and thresholds	Streaming and model API	Model-result UI	Detection demo	Planned
6 / 12-18 Oct	Feature complete	Autoencoder evaluation	Kubernetes response API	Response and filters	Feature-complete build	Planned
7 / 19-25 Oct	Hardening	Security review and metrics	Performance tuning	Regression and accessibility	Test report	Planned
8 / 26 Oct-1 Nov	Release	Final sign-off and presentation	Deployment support	Manual and final demo	v1.0 submission	Planned

6. Users and UX

6.1 Users

- Security analyst: checks and responds to alerts.
- Platform engineer: monitors Kubernetes workloads.
- Faculty evaluator: reviews the working and results.

6.2 User Flow

FLOW Login → select cluster → view live events → open alert → check evidence → approve or reject response.

6.3 User Stories

- As a security analyst, I want clear alerts with evidence so that I can quickly understand suspicious activity.
- As a platform engineer, I want pod and container details so that I can identify the affected workload.
- As an authorized responder, I want an approval step so that an unsafe action is not performed accidentally.

6.4 Accessibility

- Simple labels and readable colors.

- Keyboard-friendly dashboard.
- Severity shown using text as well as color.

7. Market and Competitors

Competitor	Product	Target users	Key features	Pricing	Strengths	Weaknesses	Our differentiator
Falco	Runtime security	Security teams	Rule-based syscall alerts	Open source	Mature CNCF project	Mostly rule-based	Rules plus ML and project dashboard
Tetragon	eBPF observability	Platform teams	Process/network visibility and enforcement	Open source	Deep Kubernetes context	More complex to deploy	Focused academic workflow
Tracee	Linux runtime security	Researchers and SOC teams	eBPF events and signatures	Open source	Rich event collection	Needs separate product layer	Sensor-to-dashboard prototype
Commercial CNAPP	Enterprise cloud security	Large organizations	Broad cloud security platform	Paid	Support and integrations	High cost and closed design	Transparent and reproducible design

7.2 Positioning

eBPF-Shield AI is an academic prototype that combines kernel monitoring, anomaly detection and Kubernetes response in one understandable system.

8. Objectives and Success Metrics

Objective	Target	KPI	Measurement
O1 - Fast event visibility	p95 event-to-dashboard time <= 3 seconds	p95 latency in seconds	Timestamped event replay
O2 - Useful detection	Detect at least 90% of selected test scenarios	Recall percentage	Labeled attack scenario matrix
O3 - Fewer false alerts	False-positive rate <= 5% on validation data	False-positive rate	Held-out normal workload data
O4 - Low overhead	Average sensor CPU < 3% and memory < 150 MB in reference test	CPU and memory usage	Baseline comparison
O5 - Clear investigation	All high/critical alerts include workload and evidence	Alert completeness percentage	Acceptance checklist

9. Key Features

Feature	Description	Priority	Dependencies	Acceptance criteria
Kernel monitoring	Collect selected process, file and network events	Must	Linux eBPF and collector	Events contain time, process and container details
Context enrichment	Add cluster, namespace, pod, container and image details	Must	Kubernetes API	Event links to the correct workload
Threat detection	Combine rules, Isolation Forest and Autoencoder	Must	Feature pipeline and trained model	Versioned score and reason appear on alerts
Live dashboard	Show live events, filters, alerts and timelines	Must	FastAPI and WebSockets	New alert appears without page refresh
Kubernetes response	Simulate, approve and execute	Should	Kubernetes RBAC	Action requires approval and

Feature	Description	Priority	Dependencies	Acceptance criteria
	isolation or termination			creates audit record
AI explanation	Summarize alert evidence in simple language	Could	Optional LLM provider	Feature fails safely when provider is unavailable

10. Architecture

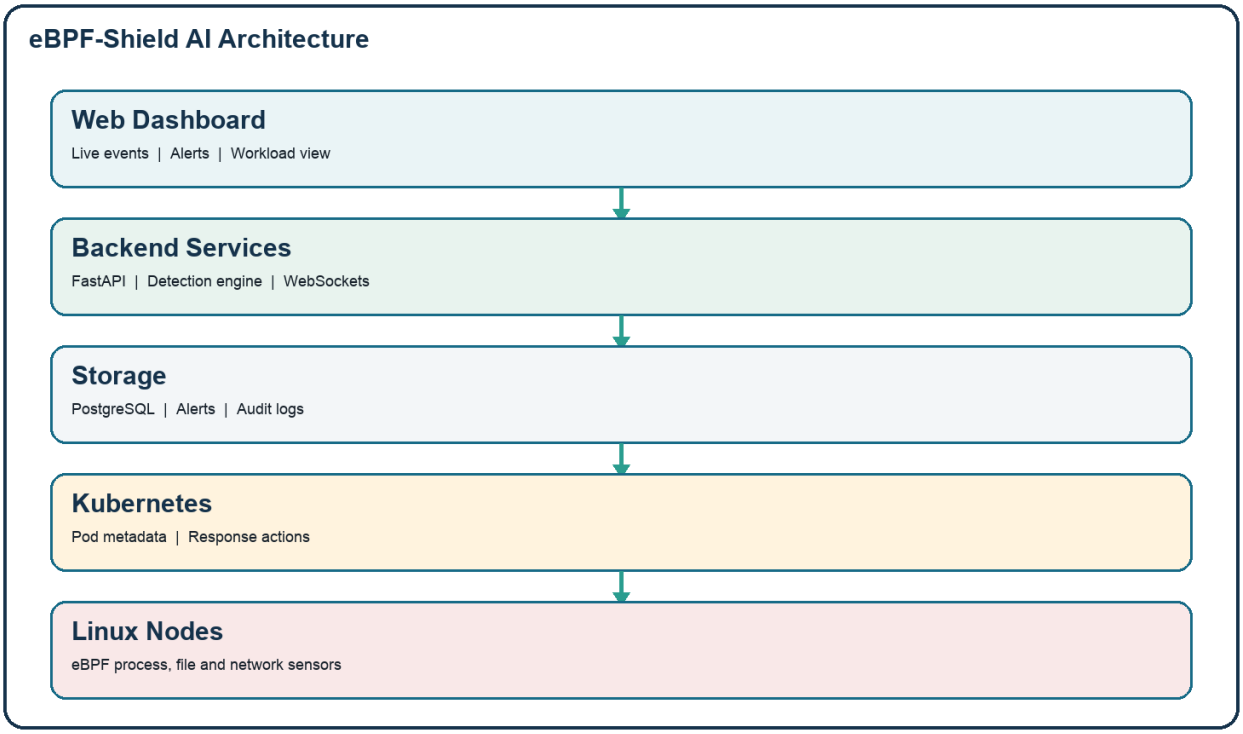


Figure 1. Architecture of eBPF-Shield AI.

Snapshots:

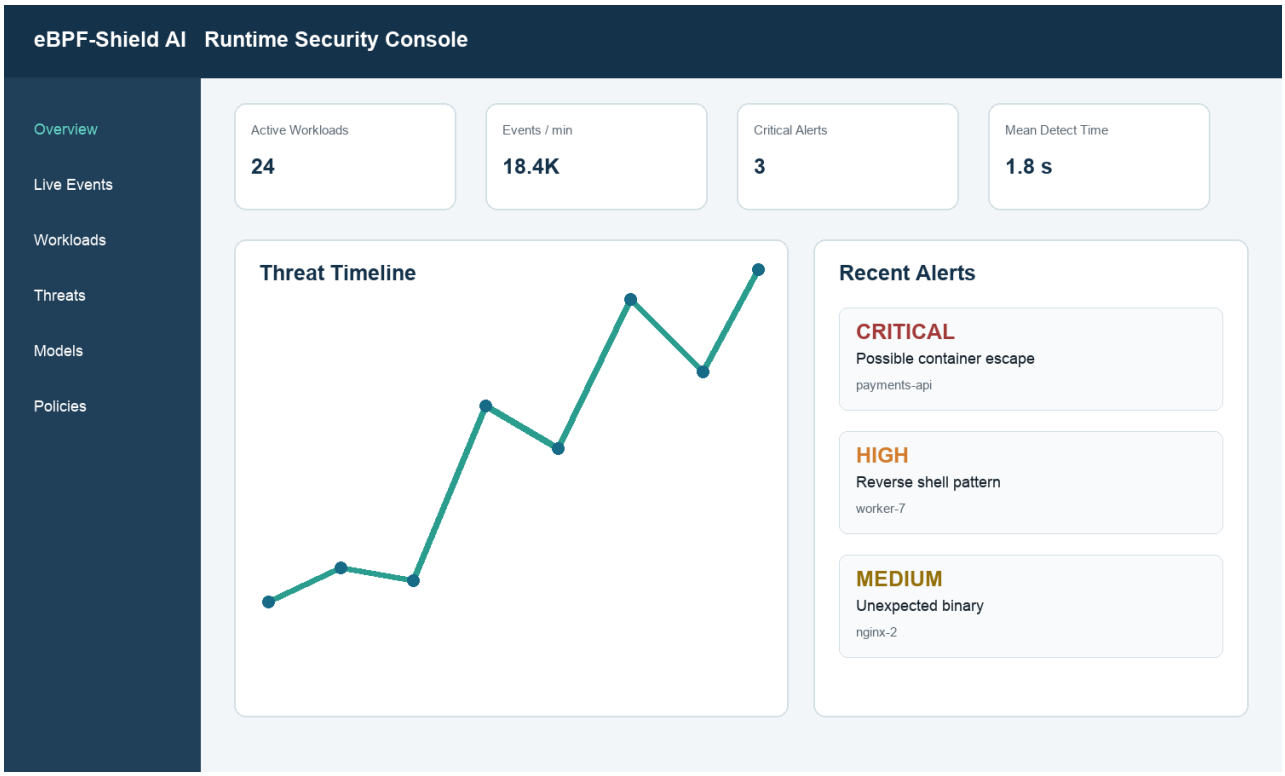


Figure 2. Runtime security dashboard.

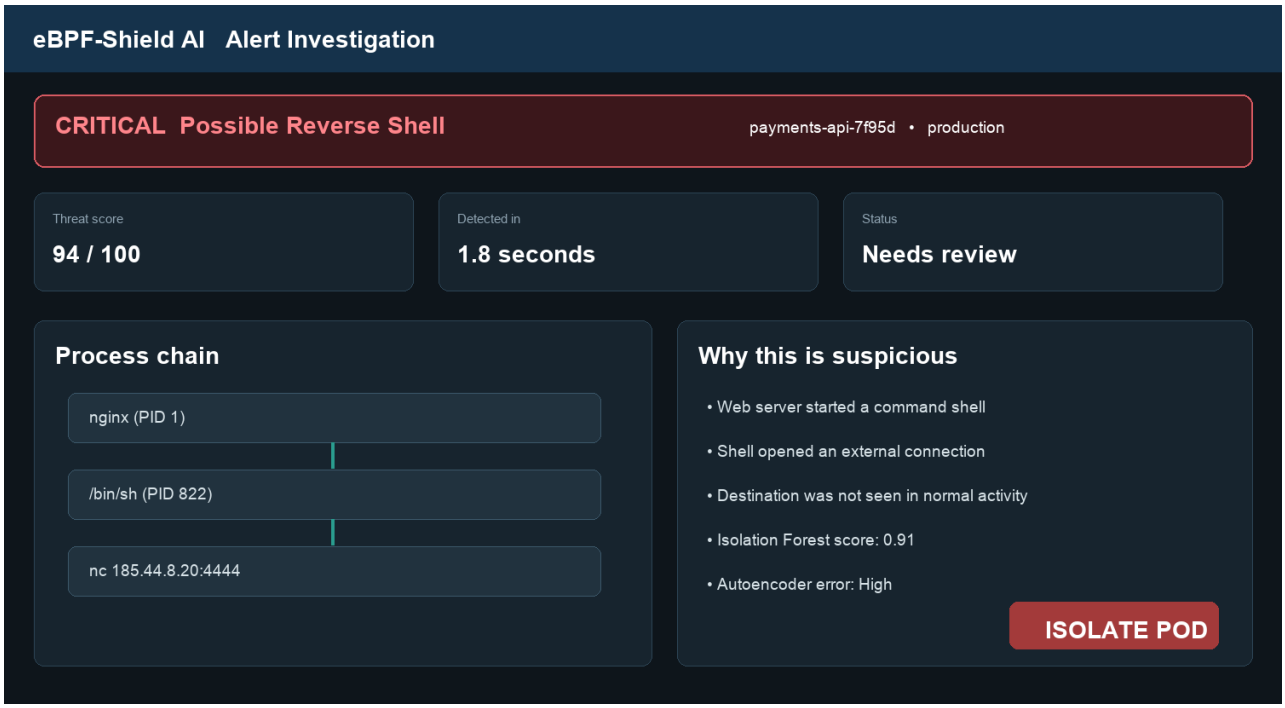


Figure 3. Alert investigation screen.

10.1 High-Level Architecture

- Client: Next.js dashboard for live events, alerts and response.
- Backend: FastAPI services receive and process events.
- Detection engine: Rules and ML models calculate the threat score.

- Database: PostgreSQL stores events, alerts, models and audit records.
- Sensor: eBPF programs collect activity from Linux nodes.
- Kubernetes: Provides workload details and controlled response.

10.2 API Spec Snapshot

Endpoint	Method	Purpose
/api/v1/events/batch	POST	Receive sensor events
/api/v1/events	GET	View runtime events
/api/v1/alerts	GET	View alerts
/api/v1/alerts/{id}	GET	View alert evidence
/api/v1/responses	POST	Request response action
/ws/v1/stream	WS	Send live updates

10.3 Configuration and Secrets

- Secrets are stored outside the source code.
- Environment variables are used for configuration.
- API keys and tokens are not committed to GitHub.
- Access is limited using roles and permissions.

11. Data Design

11.1 Data Dictionary

Entity	Important fields	Type / format	Required	Description	Notes
Workload	id, cluster, namespace, pod, container, image	UUID and strings	Yes	Identifies the monitored Kubernetes workload	Indexes on pod and container IDs
Runtime Event	id, observed_at, type, pid, ppid, executable, workload_id	UUID, timestamp, enum, integers	Yes	Stores normalized process, file or network activity	Time-based index or partition
Alert	id, severity, score, status, rule_id, model_version	UUID, enum and number	Yes	Stores the investigation result	Indexes on time, severity and status
Alert Evidence	alert_id, event_id, reason	UUIDs and text	Yes	Links an alert to the events that support it	Composite key
Model Version	id, algorithm, feature_list, threshold, artifact_hash	UUID, text and JSON	Yes	Records which model produced the score	Immutable model metadata
Response Action	id, alert_id, action, mode, actor, result, created_at	UUID, enum, text and timestamp	Yes	Records simulation, approval and execution	Append-only history
Audit Log	id, actor, action, object, timestamp	UUID, strings and timestamp	Yes	Records important user and system actions	Restricted access

11.2 Schemas and Migrations

- SQLAlchemy models define the database schema.
- Alembic manages versioned schema changes.
- UUIDs identify records and UTC is used for timestamps.
- JSONB stores optional event fields when required.

11.3 Privacy, Retention, Backup and DR

- Only required event information is stored.
- Sensitive command arguments and paths are redacted where possible.
- Raw events are retained for a limited period; alerts and audit records are kept longer.
- Backups are encrypted and restore steps are tested.

12. Technical Workflow

- 1. eBPF observes an event in the Linux kernel.
- 2. The collector adds pod and container information.
- 3. FastAPI receives and stores the event.
- 4. Rules and ML models calculate a threat score.
- 5. The dashboard displays an alert.
- 6. The user may approve a Kubernetes response.

13. Quality: NFRs and Testing

13.1 Non-functional Requirements

- Fast: Alerts should appear quickly.
- Lightweight: Sensor CPU and memory use should stay low.
- Reliable: Dropped events should be counted and reported.
- Secure: APIs and response actions require authorization.
- Usable: Alert evidence should be easy to understand.

13.2 Test Plan

Area	Type	Tools	Owner	Coverage target	Exit criteria
eBPF sensors	Unit and compatibility	Fixtures, verifier checks, kernel matrix	Aryan	Selected probes and supported kernels	Expected fields; no verifier failures
Backend API	Unit and integration	pytest, httpx, PostgreSQL test DB	Aryan	Critical endpoints and auth paths	All critical tests pass
Machine learning	Model evaluation	scikit-learn, PyTorch and replay data	Arpit	All features and selected scenarios	Precision, recall and FPR reported
Frontend	Unit, E2E and accessibility	Vitest, Playwright and axe	Ananya	Critical user flows	No serious accessibility issue
Security	Threat and dependency tests	STRIDE, SAST, image and RBAC review	Arpit	High-risk controls	No open critical/high finding
System	Scenario, load and soak	kind/minikube and replay scripts	Team	Acceptance criteria	Demo and benchmark report pass

13.3 Environments

Environment	Purpose	Main controls
Development	Local coding and synthetic event replay	Mock Kubernetes data and debug logging
Integration	Combined testing in kind/minikube	Pinned images and test credentials
Demonstration	Final repeatable project demo	Approved scenarios and read-only default

14. Security and Compliance

14.1 Threat Model (STRIDE)

Asset / boundary	Threat	STRIDE	Impact	Likelihood	Mitigation	Owner
Sensor to API	Fake or changed telemetry	Spoofing / Tampering	High	Medium	mTLS or token identity, TLS, validation and idempotency	Aryan
User account	Unauthorized access	Spoofing / Elevation	High	Low	OIDC/JWT, secure password storage and RBAC	Arpit
Event database	Evidence changed or exposed	Tampering / Disclosure	High	Low	Least-privilege roles, encryption, backups and audit	Arpit
Kubernetes response	Unauthorized isolation or termination	Elevation / DoS	High	Medium	Separate service account, allowlist, dry-run and approval	Aryan
API and WebSocket	Resource exhaustion	DoS	Medium	Medium	Authentication, rate limits, backpressure and pagination	Aryan
Logs and AI explanation	Sensitive data exposure	Disclosure	Medium	Medium	Redaction, minimization and restricted access	Ananya
Policy/model changes	Action denied later	Repudiation	Medium	Low	Actor, time and object recorded in audit log	Arpit

14.2 Authentication and Authorization

- OIDC-compatible login or short-lived JWT tokens.
- Roles: Viewer, Analyst, Responder and Administrator.
- Sensors use a separate identity from human users.
- Response permission is separate from read and ingestion permissions.

14.3 Audit and Logging

- Structured logs use event and request identifiers.
- Sensitive values are removed from application logs.
- Policy, model and response changes record the actor and time.

14.4 Compliance

- All attack simulations run only in an authorized academic test cluster.
- The design follows OWASP, NIST container guidance and Kubernetes security practices.
- The prototype does not claim formal compliance certification.

15. Delivery and Operations

15.1 Release Plan

Release	Scope	Exit gate
v0.1	Sensor-to-database flow	Process events are visible and schema is stable
v0.5	Rules, ML baseline and dashboard	Selected scenario produces a clear alert
v0.9	Response, tests and documentation	Acceptance rehearsal and security review complete

Release	Scope	Exit gate
v1.0	Final submission and reproducible demo	Mentor sign-off and archived artifacts

15.2 CI/CD and Rollback

- CI runs formatting, tests, static analysis and dependency checks.
- Builds use versioned container images and pinned dependencies.
- Deployment uses Kubernetes manifests or Helm values.
- Rollback restores the previous image, model and configuration version.

15.3 Monitoring and Alerting

Metric	Threshold / symptom	Runbook
sensor_up	Sensor unavailable for more than two minutes	RB-01 Sensor unavailable
event_drop_rate	More than 1% for five minutes	RB-02 Event drops
ingestion_latency_p95	More than three seconds for ten minutes	RB-03 Pipeline latency
alert_rate	Five times normal rate or unexpectedly zero	RB-04 Alert storm or silence
database_storage	More than 80% used	RB-05 Storage and retention
response_failure	Any failed enforced action	RB-06 Response failure

15.4 Runbooks

- Check the trigger, affected workload and sensor health.
- Inspect logs, queues, database status and recent configuration changes.
- Apply safe containment, confirm recovery and preserve evidence.
- Escalate unresolved issues and record the final result.

15.5 Communication Plan

Cadence	Audience	Information shared
Twice weekly	Project team	Progress, blockers and decisions
Weekly	Faculty mentor	Demo, metrics and risks
At each milestone	Reviewer and cluster owner	Release notes and permission changes
After a serious test issue	Team and mentor	Timeline, evidence and corrective action

16. Risks and Mitigations

Risk	Probability	Impact	Score	Mitigation	Owner	Status
Kernel incompatibility	Medium	High	6	Use CO-RE, stable tracepoints and a tested kernel matrix	Aryan	Open
High event volume	Medium	High	6	Filter, batch, apply backpressure and monitor drops	Aryan	Open

Risk	Probability	Impact	Score	Mitigation	Owner	Status
False alerts	High	Medium	6	Tune per-workload thresholds and combine rules with ML	Arpit	Open
Limited attack data	High	Medium	6	Use authorized scripted scenarios and held-out normal data	Arpit	Open
Privileged sensor risk	Medium	High	6	Use minimum capabilities, signed images and restricted access	Arpit	Open
Unsafe automated response	Low	High	3	Use dry-run, approval, allowlisted actions and RBAC	Team	Controlled
Project delay	Medium	Medium	4	Complete must-have features before optional features	Arpit	Open

17. Research and Evaluation

- Run normal workloads and collect baseline behavior.
- Perform safe attack simulations in the test cluster.
- Compare rules, Isolation Forest, Autoencoder and the combined result.
- Measure precision, recall, false alerts and detection time.
- Measure CPU, memory and event loss.
- Record model version and test settings for repeatability.

18. Appendices

18.1 Glossary

Term	Meaning
eBPF	Linux technology used to observe kernel activity
Container escape	Breaking container isolation to reach the host
Isolation Forest	ML model used to find unusual records
Autoencoder	Neural network that detects unusual reconstruction error
RBAC	Role-based access control
STRIDE	Method used to study security threats

18.2 References

- Linux Kernel BPF Documentation — https://docs.kernel.org/bpf/
- Kubernetes Security Documentation — https://kubernetes.io/docs/concepts/security/
- NIST SP 800-190 — Application Container Security Guide
- Falco Documentation — https://falco.org/docs/
- Tetragon Documentation — https://tetragon.io/docs/
- MITRE ATT&CK for Containers — https://attack.mitre.org/matrices/enterprise/containers/
- FastAPI Documentation — https://fastapi.tiangolo.com/
- Liu, Ting and Zhou, "Isolation Forest," IEEE ICDM, 2008.